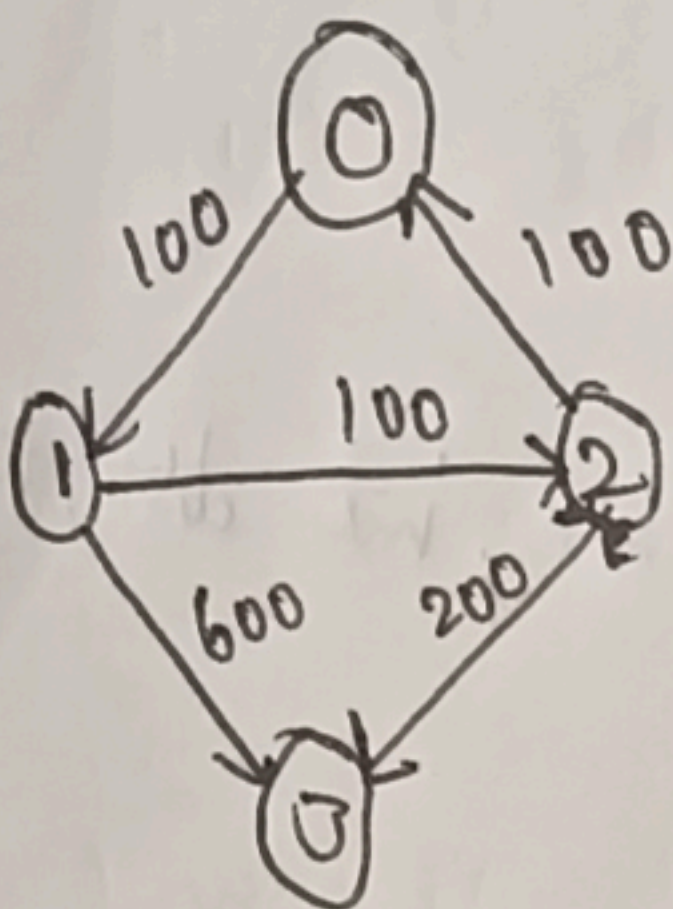


Name - Divij Mahajan

UID - 20BCS12775

Class - 20BCS-KRG-1-A

Q1.



- Algo:

1. Build Adjacency list from the edges.

2. Make a priority-queue (min-heap).

3. Initialise queue where we will insert (cost, node, steps).

• Push (0, src, 0)

4. Make distance array

• set $\text{dist}[\text{src}] = 0$

• set other dist as ∞ .

5. while priority-queue not empty:

• Pop (cost, node, steps)

• if steps > K continue

• for each neighbour:

if $\text{cost} + \text{edgecost} < \text{dist}[\text{nbe}]$

update $\text{dist}[\text{nbe}]$

push (newCost, nbe, steps + 1)

5. if (dist[destination] == ∞)

return -1;

else

return dist[destination];

→ code:

int solve(int n, vector<vector<int>>& edges, int src, int dst, int k)

~~vector<vector<int>>~~

vector<vector<pair<int, int>>> adj(n);

for (auto edge : edges) {

int u = edge[0];

int v = edge[1];

int w = edge[2];

adj[u].push_back({v, w});

}

priority_queue<tuple<int, int, int>, vector<tuple<int, int, int>>,
greater<>> > pq;

pq.push({0, src, 0});

vector<int> dist(n, INT_MAX);

dist[src] = 0;

while (!pq.empty()) {

auto [cost, u, steps] = pq.top();

pq.pop();

if (steps > K) continue;

if (~~node~~ == dest) return cost;

dist[u] = steps;

for (auto & mbe : adj[u]) {

int v = mbe.first;

int w = mbe.second;

if (cost + w < dist[v])

pq.push({cost + w, v, steps + 1});

}

}

return -1;

}